Prof. Dr. Thomas Schultz
Jonathan Lennartz (jlen@uni-bonn.de)
Annika Mikliss (amikliss@uni-bonn.de)

University of Bonn
B-IT/Computer Science II
May 29, 2025

Summer term 2025

Visual Data Analysis

Assignment Sheet 9

Solution has to be submitted via eCampus by June 6, 2025, 10:00 a.m.

If you have questions concerning the exercises, please use the forum on eCampus.

- Please work on this exercise in **small groups** of 3 students. Submit each solution only once, but clearly indicate who contributed to it by forming a team in eCampus. Remember that all team members have to be able to explain all answers.
- Please submit your answers in PDF format, and your scripts as *.py/*.ipynb files. If you are using Jupyter notebook, please also export your scripts and results as PDF.

Exercise 1 (Introduction to ParaView, *13 Points*)

In this exercise we introduce you to ParaView, a cross-platform data analysis and visualization application that implements most algorithms we will discuss in the next few weeks. With ParaView you can quickly visualize spatial data from simulations, medicine, or engineering. You can download the recent version of ParaView from `https://www.paraview.org/download/`. Please send us both a screenshot and the corresponding Paraview state (`*.pvsm` file) of your solution.

Hint: On Windows, an MPI and a non-MPI version of ParaView are available. The non-MPI version is sufficient for our purposes and should work "out of the box" after running the installer. Getting the MPI version to run requires installing additional libraries.

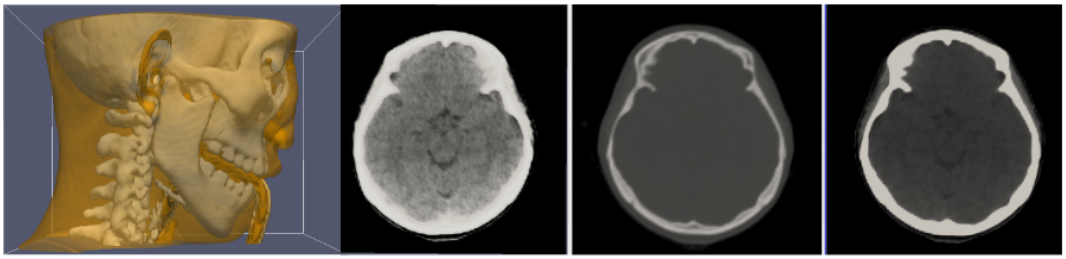


Figure 1: Nested 3D surfaces (left) and brain slices with different colormaps (right).

a) Load the file `headsq.vti` into a ParaView pipeline. What is the grid type of this dataset? What is the type and range of its values? (2P)

b) A basic and widely used way to visualize 3D CT data is via 2D slices. Different mappings from Hounsfield units to grayscale are used depending on the structures of interest. To examine brain, bone, or potential intracranial hemorrhage, common ranges ("windows") are $[0, 80]$, $[-770, 1730]$ and $[-85, 265]$ Hounsfield units, respectively. Create three corresponding images, which should look similar to the ones in Figure 1. (6P)
Hint: The values in the given input file correspond to shifted Hounsfield units: 1024 has been added in order to store them as unsigned values.

c) Also create a 3D visualization showing two nested surfaces as in the left part of Figure 1: The outer surface should show the skin, and it should be rendered in a different color as well as semi-transparently in order to reveal an inner surface, which should visualize the bones. (5P)
Hint: This can be achieved with isosurfaces, which are available via ParaView's `Contour` filter.

Exercise 2 (Visualization Pipeline in Python, *12+4 Bonus Points*)

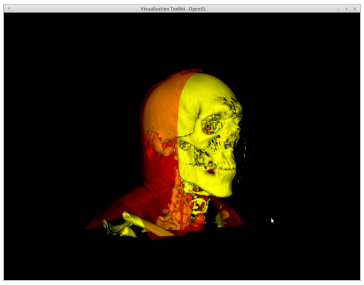


Figure 2: Final result of the Python script.

In this exercise, you will use VTK's Python bindings to implement a simple visualization pipeline for volumetric data. The goal is to read the file `head.vti` (available from eCampus) and create a visualization similar to the one in Fig. 2. On eCampus, we provide a framework that already implements some of the functions you will need. We have marked the areas where you have to add code yourself with "<Insert Code>".

a) Finish the function "task1": Create a bounding box for the input data using an outline contour filter (vtkOutlineFilter) and set the color of the bounding box to green.
b) Finish the function "task2": Create a contour for the bones using vtkContourFilter and set the color to yellow. You may have to try multiple isovalues. Which one is suitable for extracting the bone structure?
c) Finish the function "task3": Create a contour for the skin, set the color to red and make it transparent. Also add the clipping plane using vtkPlane.
d) In the main function, implement the part which disables the visibility of the bounding box when ShowBBox is not set, so that the bounding box is only shown when the '-b true' parameter is used at the command line.

Please submit a screenshot of your final result and your Python script.

Hints: You can find the documentation for VTK's Python wrappers at https://docs.vtk.org/en/latest/advanced/PythonWrappers.html. There is also a large collection of useful examples for how to use VTK via Python at https://examples.vtk.org/site/Python/.

Bonus Task: Add another command line parameter "-a true" that enables an animation in which the camera is rotated around the dataset.

Hint: In order to do the animation, you will have to create a function that will be executed automatically after a given amount of time (i.e., triggered by a timer). In this function, you will have to get the active camera that is used in the renderer and rotate it. You will need to call several Get<SOMETHING>() functions starting from the interactor.

Exercise 3 (Mapping Between Index and World Space, *5 Points*)

Assume a uniform grid with origin at $\mathbf{p} = (1, -1, 2)^T$, axis directions $\mathbf{e} = 1/\sqrt{2}(1, 0, 1)^T$, $\mathbf{f} = (0, 1, 0)^T$, $\mathbf{g} = 1/\sqrt{2}(1, 0, -1)^T$, and axis spacings $dx = 1$, $dy = 2$, and $dz = 1$.

Hint: You can use a computer (e.g., Python) to do the computations in this task. It is acceptable if you specify rounded results.

a) Specify the affine transformation matrix from index space coordinates (i, j, k) to world space coordinates (x, y, z). Then specify the corresponding mapping for transferring world space coordinates (x, y, z) back to index space coordinates (i, j, k). (3P)

b) Use the transformation matrix to determine the world space position of the voxel with index coordinates $(2, 1, 5)$, and the displacement vector in world space that corresponds to incrementing the first index space coordinate by one voxel, the second one by two voxels, and the third one by minus one voxel. Which of the answers changes if we move the origin $\mathbf{p}$ and why? (2P)

Exercise 4 (Finite Differences, *5 Points*)

a) Use the Taylor expansion to show that the forward finite difference scheme on a uniform grid with stepsize h is first-order accurate (i.e., the approximation error decreases linearly with decreasing stepsize h). (3P)

b) Use the same strategy to show that the central finite difference scheme is second-order accurate (i.e., the approximation error decreases with h^2). (2P)

Exercise 5 (Asymptotic Decider, *6+3 Bonus Points*)

a) According to Chapter 9, the direct way to evaluate the asymptotic decider involves re-writing the standard equation for bilinear interpolation into a form that allows us to read off the intersection point of the asymptotes and the function value at that point. Specify explicit formulas to compute η, x_0, y_0, and γ from $f_{i,j}$, $f_{i+1,j}$, $f_{i,j+1}$, and $f_{i+1,j+1}$. (4P)

b) In cases which require the asymptotic decider, do we ever have to divide by zero in the equations you derived in a)? Why? (2P)

c) Unfortunately, the scientific paper that introduced the asymptotic decider contained a serious typo. You can download this paper from eCampus. We will give you a bonus point if you identify the equation in that paper where the equivalent of γ (in our notation) is computed, and two more if you tell us what is wrong with this equation, and how to fix it.

Exercise 6 (Marching Cubes, *9 Points*)

Given the cells in Figure 3, apply the Marching Cubes algorithm for the isovalue $c = 0$ by drawing the resulting triangles of the isosurface. Indicate which of the 15 base cases shown in Figure 4 has to be used to connect the edge intersections in each case. Mark the faces of the cells that result in ambiguous configurations and apply the asymptotic decider where needed.

Good Luck!

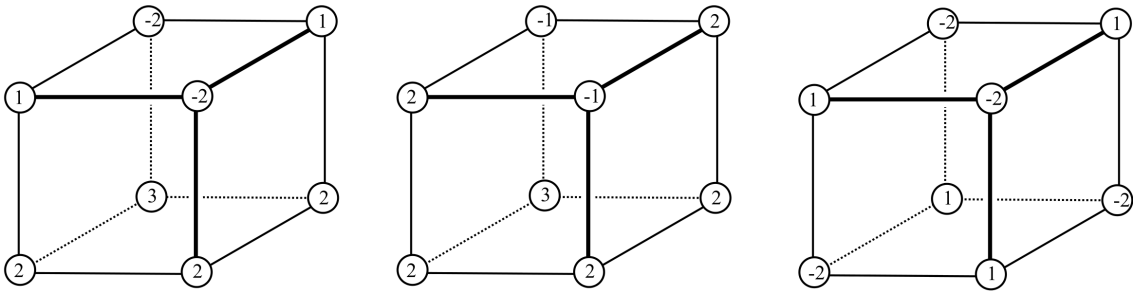


Figure 3: Your task is to apply Marching Cubes to these cells.

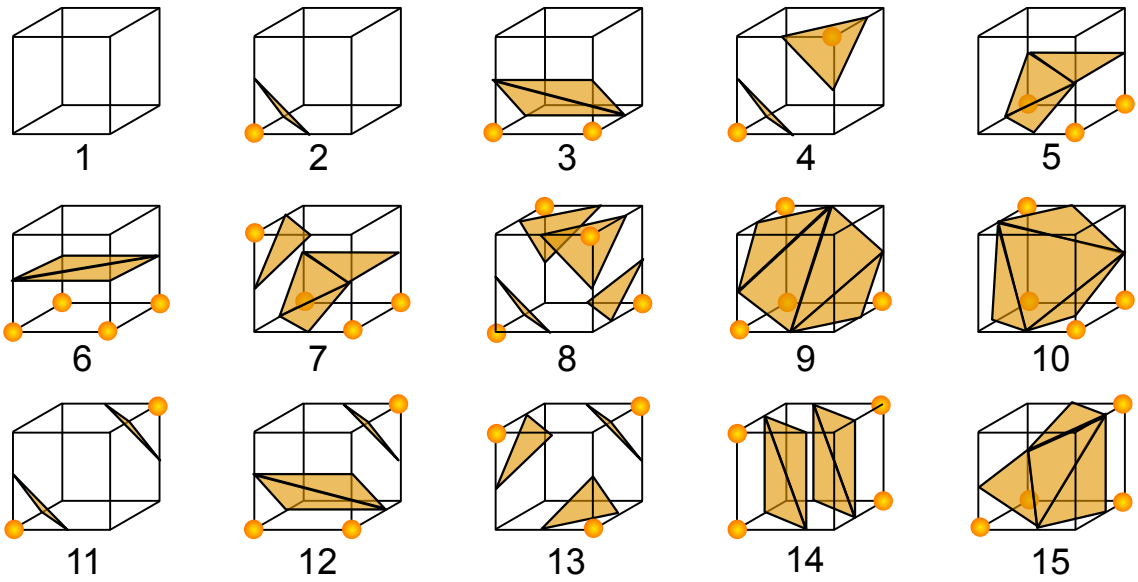


Figure 4: Marching Cubes cases. Please remember that, depending on the outcome of the decider, some cases might result in different triangles than shown in this table.